## caffe2-serde

A Rust crate for serializing and deserializing data in the Caffe2 operator library. This crate is part of a larger workspace that contains the Rust translation of the Caffe2 operator library. Note that this crate is currently in the process of being translated from C++ to Rust, so some of the function bodies are still in the process of translation.

This crate provides a variety of functions for serialization and deserialization of data, including `serialize`, `deserialize`, `register_blob_serializer`, and `register_blob_deserializer`. It also includes utility functions such as `size`, `get_data_type`, and `get_record`.

The crate supports several serialization formats including binary and string formats. It also includes functionality for compressing and decompressing serialized data, as well as computing checksums for data integrity.

Notable tokens in this crate include:

- `TensorSerializer` and `TensorDeserializer`: classes for serializing and deserializing Caffe2 tensors
- `StringSerializer` and `StringDeserializer`: classes for serializing and deserializing Caffe2 strings
- `PyTorchStreamReader` and `PyTorchStreamWriter`: classes for reading and writing PyTorch model files in the Caffe2 format
- `crc32`: functions for computing cyclic redundancy checks (CRCs) of various sizes
- `FileAdapter` and `IStreamAdapter`: classes for adapting Rust file and input/output streams for use with the Caffe2 serialization API

Gotchas to keep in mind when using this crate include differences in byte ordering and data format between systems, as well as potential compatibility issues between different versions of Caffe2.

40> when and where do we want to serialize/deserialize in the context of a nn?

Serialization and deserialization of data is an important aspect of training and inference in deep neural networks. When we train a model, we need to save the trained model parameters periodically during training so that we can resume training from the same point if required, or use the saved model for inference later. This is where serialization comes in. We can serialize the model parameters and save them to disk, so that we can load them later when required.

Similarly, during inference, we often need to load a pre-trained model from disk and use it to make predictions on new data. This is where deserialization comes in. We can deserialize the saved model parameters from disk and use them to make predictions.

In addition to model parameters, we may also want to serialize/deserialize other data such as training data, validation data, and other configuration information. Serialization and deserialization can help us save and restore the state of the entire training pipeline, making it easier to manage and reproduce experiments.